

**LAPORAN PRAKTIKUM  
PEMROGRAMAN MOBILE  
MODUL 5**



**Connect to the Internet**

**Oleh:**

**Harry Pratama Yunus    NIM. 2310817210010**

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS TEKNIK  
UNIVERSITAS LAMBUNG MANGKURAT  
JUNI 2025**

**LEMBAR PENGESAHAN**  
**LAPORAN PRAKTIKUM PEMROGRAMAN I**  
**MODUL 5**

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Harry Pratama Yunus  
NIM : 2310817210010

Menyetujui,  
Asisten Praktikum

Mengetahui,  
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar  
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom  
NIP. 198205082008011010

## DAFTAR ISI

|                         |    |
|-------------------------|----|
| LEMBAR PENGESAHAN ..... | 2  |
| DAFTAR ISI .....        | 3  |
| DAFTAR GAMBAR.....      | 4  |
| DAFTAR TABEL .....      | 5  |
| SOAL 1.....             | 6  |
| A. Source Code.....     | 6  |
| B. Output Program ..... | 29 |
| C. Pembahasan .....     | 31 |
| Tautan Git .....        | 48 |

## DAFTAR GAMBAR

|                                             |    |
|---------------------------------------------|----|
| Gambar 1 Screenshot output light mode ..... | 29 |
| Gambar 2 Screenshot output dark mode.....   | 30 |

## DAFTAR TABEL

|                                                     |    |
|-----------------------------------------------------|----|
| Table 1 Source code MovieDao.kt .....               | 6  |
| Table 2 Source code MovieDatabase.kt .....          | 7  |
| Table 3 Source code MovieEntity.kt .....            | 7  |
| Table 4 Source code PreferencesManager.kt.....      | 8  |
| Table 5 Source code ApiService.kt .....             | 8  |
| Table 6 Source code MovieDao.kt .....               | 9  |
| Table 7 Source code RetrofitInstance.kt.....        | 9  |
| Table 8 Source code MovieRepositoryImplkt .....     | 10 |
| Table 9 Source code Movie.kt .....                  | 11 |
| Table 10 Source code MovieRepository.kt .....       | 12 |
| Table 11 Source code MovieUseCase.kt .....          | 12 |
| Table 12 Source code DetailScreen.kt .....          | 12 |
| Table 13 Source code MainScreen.kt .....            | 16 |
| Table 14 Source code MovieCard.kt .....             | 18 |
| Table 15 Source code TopBarApp.kt .....             | 22 |
| Table 16 Source code MainActivity.kt .....          | 22 |
| Table 17 Source code MovieViewModel.kt .....        | 25 |
| Table 18 Source code MovieViewModelFactory.kt.....  | 26 |
| Table 19 Source code ThemeViewModel.kt .....        | 26 |
| Table 20 Source code ThemeViewModelFactory.kt ..... | 27 |
| Table 21 Source code NetworkResult.kt .....         | 27 |

## SOAL 1

### Soal Praktikum:

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- Gunakan KotlinX Serialization sebagai library JSON.
- Gunakan library seperti Coil atau Glide untuk image loading.
- API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

### A. Source Code

#### COMPOSE :

#### MovieDao.kt

Table 1 Source code MovieDao.kt

|    |                                                           |
|----|-----------------------------------------------------------|
| 1  | package com.pemrogamanmobile.movielist.data.local.dao     |
| 2  |                                                           |
| 3  | import androidx.room.Dao                                  |
| 4  | import androidx.room.Insert                               |
| 5  | import androidx.room.OnConflictStrategy                   |
| 6  | import androidx.room.Query                                |
| 7  | import                                                    |
|    | com.pemrogamanmobile.movielist.data.local.entity.MovieEnt |
| 8  | ity                                                       |
| 9  | @Dao                                                      |
| 10 | interface MovieDao {                                      |
| 11 | @Insert(onConflict = OnConflictStrategy.REPLACE)          |
| 12 | suspend fun insertMovies(movies: List<MovieEntity>)       |
| 13 |                                                           |
| 14 | @Query("SELECT * FROM movies")                            |
| 15 | suspend fun getAllMovies(): List<MovieEntity>             |

|    |                              |
|----|------------------------------|
| 16 |                              |
| 17 | @Query("DELETE FROM movies") |
| 18 | suspend fun clearMovies()    |
| 19 | }                            |

## MovieDatabase.kt

Table 2 Source code MovieDatabase.kt

|    |                                                          |
|----|----------------------------------------------------------|
| 1  | package com.pemrogamanmobile.movielist.data.local.db     |
| 2  |                                                          |
| 3  | import android.content.Context                           |
| 4  | import androidx.room.Database                            |
| 5  | import androidx.room.Room                                |
| 6  | import androidx.room.RoomDatabase                        |
| 7  | import                                                   |
|    | com.pemrogamanmobile.movielist.data.local.dao.MovieDao   |
|    | import com.pemrogamanmobile.movielist.data.local.entity. |
|    | MovieEntity                                              |
| 8  |                                                          |
| 9  | @Database(entities = [MovieEntity::class],               |
| 10 | version = 1, exportSchema = false)                       |
| 11 | abstract class MovieDatabase : RoomDatabase() {          |
| 12 | abstract fun movieDao(): MovieDao                        |
| 13 |                                                          |
| 14 | companion object {                                       |
| 15 | @Volatile                                                |
| 16 | private var INSTANCE: MovieDatabase? = null              |
| 17 |                                                          |
| 18 | fun getDatabase(context: Context): MovieDatabase         |
|    | =                                                        |
| 19 | INSTANCE ?: synchronized(this) {                         |
| 20 | INSTANCE ?: Room.databaseBuilder(                        |
| 21 | context.applicationContext,                              |
| 22 | MovieDatabase::class.java,                               |
| 23 | "movie_db"                                               |
| 24 | ).fallbackToDestructiveMigration()                       |
|    | .build().also { INSTANCE = it }                          |
| 25 | }                                                        |
| 26 | }                                                        |
| 27 | }                                                        |

## MovieEntity.kt

Table 3 Source code MovieEntity.kt

|   |                                                          |
|---|----------------------------------------------------------|
| 1 | package com.pemrogamanmobile.movielist.data.local.entity |
|---|----------------------------------------------------------|

|    |                                 |
|----|---------------------------------|
| 2  |                                 |
| 3  | import androidx.room.Entity     |
| 4  | import androidx.room.PrimaryKey |
| 5  |                                 |
| 6  | @Entity(tableName = "movies")   |
| 7  | data class MovieEntity(         |
| 8  | @PrimaryKey val id: Int,        |
| 9  | val title: String,              |
| 10 | val overview: String,           |
| 11 | val posterPath: String,         |
| 12 | val releaseDate: String         |
| 13 | )                               |

### PreferencesManager.kt

Table 4 Source code PreferencesManager.kt

|    |                                                         |
|----|---------------------------------------------------------|
| 1  | package com.pemrogamanmobile.movielist.data.preferences |
| 2  |                                                         |
| 3  | import android.content.Context                          |
| 4  | import android.content.SharedPreferences                |
| 5  |                                                         |
| 6  | class PreferencesManager(context: Context) {            |
| 7  | private val prefs: SharedPreferences =                  |
|    | context.getSharedPreferences("movie_prefs",             |
|    | Context.MODE_PRIVATE)                                   |
| 8  |                                                         |
| 9  | fun saveThemeSetting(isDarkMode: Boolean) {             |
| 10 | prefs.edit().putBoolean("dark_mode",                    |
|    | isDarkMode).apply()                                     |
| 11 | }                                                       |
| 12 |                                                         |
| 13 | fun isDarkMode(): Boolean =                             |
|    | prefs.getBoolean("dark_mode", false)                    |
| 14 | }                                                       |

### ApiService.kt

Table 5 Source code ApiService.kt

|   |                                                        |
|---|--------------------------------------------------------|
| 1 | package com.pemrogamanmobile.movielist.data.remote.api |
| 2 |                                                        |
| 3 | import com.pemrogamanmobile.movielist.data.remote.dto. |
|   | MovieResponseDto                                       |
| 4 | import retrofit2.http.GET                              |
| 5 | import retrofit2.http.Query                            |
| 6 |                                                        |
| 7 | interface ApiService {                                 |
| 8 | @GET("movie/popular")                                  |



|    |                                  |
|----|----------------------------------|
| 9  | suspend fun getPopularMovies (   |
| 10 | @Query("api_key") apiKey: String |
| 11 | ) : MovieResponseDto             |
| 12 | }                                |

## MovieDto.kt

Table 6 Source code MovieDao.kt

|    |                                                        |
|----|--------------------------------------------------------|
| 1  | package com.pemrogamanmobile.movielist.data.remote.dto |
| 2  |                                                        |
| 3  | import kotlinx.serialization.SerialName                |
| 4  | import kotlinx.serialization.Serializable              |
| 5  |                                                        |
| 6  | @Serializable                                          |
| 7  | data class MovieDto(                                   |
| 8  | val id: Int,                                           |
| 9  | val title: String,                                     |
| 10 | val overview: String,                                  |
| 11 | @SerialName("poster_path") val posterPath: String,     |
| 12 | @SerialName("release_date") val releaseDate: String    |
| 13 | )                                                      |
| 14 |                                                        |
| 15 | @Serializable                                          |
| 16 | data class MovieResponseDto(                           |
| 17 | val results: List<MovieDto>                            |
| 18 | )                                                      |

## RetrofitInstance.kt

Table 7 Source code RetrofitInstance.kt

|    |                                                        |
|----|--------------------------------------------------------|
| 1  | package com.pemrogamanmobile.movielist.data.remote     |
| 2  |                                                        |
| 3  | import com.jakewharton.retrofit2.converter.kotlinx.    |
|    | serialization.asConverterFactory                       |
| 4  | import com.pemrogamanmobile.movielist.data.remote.api. |
|    | ApiService                                             |
| 5  | import kotlinx.serialization.json.Json                 |
| 6  | import okhttp3.MediaType.Companion.toMediaType         |
| 7  | import retrofit2.Retrofit                              |
| 8  |                                                        |
| 9  | object RetrofitInstance {                              |
| 10 | private const val BASE_URL =                           |
|    | "https://api.themoviedb.org/3/"                        |
| 11 |                                                        |
| 12 | private val retrofit by lazy {                         |

|    |                                                       |
|----|-------------------------------------------------------|
| 13 | Retrofit.Builder()                                    |
| 14 | .baseUrl (BASE_URL)                                   |
| 15 | .addConverterFactory                                  |
|    | (Json { ignoreUnknownKeys = true }.asConverterFactory |
|    | ("application/json".toMediaType()))                   |
| 16 | .build()                                              |
| 17 | }                                                     |
| 18 |                                                       |
| 19 | val api: ApiService by lazy {                         |
| 20 | retrofit.create(ApiService::class.java)               |
| 21 | }                                                     |
| 22 | }                                                     |

## MovieRepositoryImpl.kt

Table 8 Source code MovieRepositoryImpl.kt

|    |                                                        |
|----|--------------------------------------------------------|
| 1  | package com.pemrogamanmobile.movielist.data.repository |
| 2  |                                                        |
| 3  | import                                                 |
|    | com.pemrogamanmobile.movielist.data.local.dao.MovieDao |
| 4  | import com.pemrogamanmobile.movielist.data.remote.api. |
|    | ApiService                                             |
| 5  | import com.pemrogamanmobile.movielist.domain.model.    |
|    | Movie                                                  |
| 6  | import com.pemrogamanmobile.movielist.domain.          |
|    | repository.MovieRepository                             |
| 7  | import                                                 |
|    | com.pemrogamanmobile.movielist.utils.NetworkResult     |
| 8  | import kotlinx.coroutines.flow.Flow                    |
| 9  | import kotlinx.coroutines.flow.flow                    |
| 10 |                                                        |
| 11 | class MovieRepositoryImpl(                             |
| 12 | private val api: ApiService,                           |
| 13 | private val dao: MovieDao                              |
| 14 | ) : MovieRepository {                                  |
| 15 | override fun                                           |
|    | getPopularMovies(): Flow<NetworkResult                 |
|    | <List<Movie>>> = flow {                                |
| 16 | emit(NetworkResult.Loading)                            |
| 17 | try {                                                  |
| 18 | val response = api.                                    |
|    | getPopularMovies("a9936cbd33c74fbfa33d892ad498af3c")   |
| 19 | val movies = response.results.map {                    |
| 20 | Movie(it.id, it.title,                                 |
|    | it.overview, it.posterPath, it.releaseDate)            |
| 21 | }                                                      |

|    |                                                                       |
|----|-----------------------------------------------------------------------|
| 22 | dao.clearMovies()                                                     |
| 23 | dao.insertMovies(movies.map { entity ->                               |
| 24 | com.pemrogamanmobile.movielist.data.local.<br>entity.<br>MovieEntity( |
| 25 | entity.id,                                                            |
| 26 | entity.title,                                                         |
| 27 | entity.overview,                                                      |
| 28 | entity.posterPath,                                                    |
| 29 | entity.releaseDate                                                    |
| 30 | )                                                                     |
| 31 | })                                                                    |
| 32 | emit(NetworkResult.Success(movies))                                   |
| 33 | } catch (e: Exception) {                                              |
| 34 | val cached = dao.getAllMovies().map {                                 |
| 35 | Movie(it.id, it.title,                                                |
| 36 | it.overview, it.posterPath, it.releaseDate)                           |
| 37 | }<br>if (cached.isNotEmpty()) {                                       |
| 38 | emit(NetworkResult.Success(cached))                                   |
| 39 | } else {                                                              |
| 40 | emit(NetworkResult.Error                                              |
| 41 | (e.message ?: "Unknown Error"))                                       |
| 42 | }                                                                     |
| 43 | }                                                                     |
| 44 | }                                                                     |

## Movie.kt

Table 9 Source code Movie.kt

|   |                                                     |
|---|-----------------------------------------------------|
| 1 | package com.pemrogamanmobile.movielist.domain.model |
| 2 |                                                     |
| 3 | data class Movie(                                   |
| 4 | val id: Int,                                        |
| 5 | val title: String,                                  |
| 6 | val overview: String,                               |
| 7 | val posterPath: String,                             |
| 8 | val releaseDate: String                             |
| 9 | )                                                   |

## MovieRepository.kt

Table 10 Source code MovieRepository.kt

|   |                                                          |
|---|----------------------------------------------------------|
| 1 | package com.pemrogamanmobile.movielist.domain.repository |
| 2 |                                                          |
| 3 | import com.pemrogamanmobile.movielist.domain.model.Movie |
| 4 | import                                                   |
|   | com.pemrogamanmobile.movielist.utils.NetworkResult       |
| 5 | import kotlinx.coroutines.flow.Flow                      |
| 6 |                                                          |
| 7 | interface MovieRepository {                              |
| 8 | fun                                                      |
|   | Flow<NetworkResult<List<Movie>>> getPopularMovies() :    |
| 9 | }                                                        |

## MovieUseCase.kt

Table 11 Source code MovieUseCase.kt

|    |                                                          |
|----|----------------------------------------------------------|
| 1  | package com.pemrogamanmobile.movielist.domain.usecase    |
| 2  |                                                          |
| 3  | import com.pemrogamanmobile.movielist.domain.model.Movie |
| 4  | import com.pemrogamanmobile.movielist.domain.repository. |
|    | MovieRepository                                          |
| 5  | import                                                   |
|    | com.pemrogamanmobile.movielist.utils.NetworkResult       |
| 6  | import kotlinx.coroutines.flow.Flow                      |
| 7  |                                                          |
| 8  | class MovieUseCase(private val repository:               |
|    | MovieRepository) {                                       |
| 9  | fun                                                      |
|    | Flow<NetworkResult<List<Movie>>> getPopularMovies() :    |
| 10 | return repository.getPopularMovies()                     |
| 11 | }                                                        |
| 12 | }                                                        |

## DetailScreen.kt

Table 12 Source code DetailScreen.kt

|   |                                                        |
|---|--------------------------------------------------------|
| 1 | package                                                |
|   | com.pemrogamanmobile.movielist.presentation.ui.screen  |
| 2 |                                                        |
| 3 | import androidx.compose.foundation.Image               |
| 4 | import androidx.compose.foundation.layout.*            |
| 5 | import androidx.compose.foundation.rememberScrollState |
| 6 | import                                                 |
|   | androidx.compose.foundation.shape.RoundedCornerShape   |
| 7 | import androidx.compose.foundation.verticalScroll      |

```

8 import androidx.compose.material3.*
9 import androidx.compose.runtime.*
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.draw.clip
13 import androidx.compose.ui.layout.ContentScale
14 import androidx.compose.ui.text.font.FontWeight
15 import androidx.compose.ui.unit.dp
16 import androidx.compose.ui.unit.sp
17 import androidx.navigation.NavController
18 import coil.compose.rememberAsyncImagePainter
19 import
    com.pemrogamanmobile.movielist.presentation.viewmodel.
    MovieViewModel
20 import
    com.pemrogamanmobile.movielist.utils.NetworkResult
21
22 @Composable
23 fun DetailScreen(
24     viewModel: MovieViewModel,
25     movieId: Int,
26     isDarkMode: Boolean,
27     onToggleTheme: () -> Unit
28 ) {
29     val moviesResult by
    viewModel.movies.collectAsState()
30     val scrollState = rememberScrollState()
31     val colorScheme = MaterialTheme.colorScheme
32
33     Scaffold(
34         topBar = {
35             TopAppBar(
36                 title = "Detail Movie",
37                 isDarkMode = isDarkMode,
38                 onToggleTheme = onToggleTheme
39             )
40         }
41     ) { paddingValues ->
42         when (moviesResult) {
43             is NetworkResult.Loading -> {
44                 Box(modifier = Modifier
45                     .fillMaxSize()
46                     .padding(paddingValues),
47                     contentAlignment = Alignment.Center
48                 ) {
49                     CircularProgressIndicator(color =
    colorScheme.primary)

```

```

50         }
51     }
52
53     is NetworkResult.Success -> {
54         val movie = (moviesResult as
NetworkResult.Success).data.find { it.id == movieId }
55         if (movie != null) {
56             Surface(
57                 modifier = Modifier
58                     .fillMaxSize()
59                     .padding(paddingValues),
60                 color = colorScheme.background
61             ) {
62                 Column(
63                     modifier = Modifier
64                         .padding(16.dp)
65
66                     .verticalScroll(scrollState)
67                     ) {
68                     Image(
69                         painter =
rememberAsyncImagePainter
("https://image.tmdb.org/t/p/w500${movie.posterPath}"),
70                         contentDescription =
movie.title,
71                         modifier = Modifier
72                             .fillMaxWidth()
73                             .height(400.dp)
74
75                         .clip(RoundedCornerShape(6.dp)),
76                         contentScale =
ContentScale.Crop
77                     )
78                     Column(modifier =
Modifier.fillMaxWidth()) {
79                         Row(
80                             modifier =
horizontalArrangement = Arrangement.SpaceBetween
81                             ) {
82                             Text(
83                                 text =
movie.title,
84                                 style =
MaterialTheme.typography.headlineSmall.copy(

```

|     |                                             |                      |   |
|-----|---------------------------------------------|----------------------|---|
| 85  |                                             | color                | = |
| 86  | colorScheme.onBackground                    |                      |   |
| 87  |                                             | ),                   |   |
| 87  |                                             | modifier             | = |
| 88  | Modifier.weight(1f)                         |                      |   |
| 89  |                                             | )                    |   |
| 90  |                                             | Text(                |   |
| 91  | movie.releaseDate,                          | text                 | = |
| 92  |                                             | style                | = |
| 93  | MaterialTheme.typography.bodyMedium.copy(   |                      |   |
| 94  |                                             | color                | = |
| 95  | colorScheme.onBackground.copy(alpha = 0.6f) |                      |   |
| 96  |                                             | ),                   |   |
| 97  |                                             | modifier             | = |
| 98  | Modifier.align(Alignment.CenterVertically)  |                      |   |
| 99  |                                             | )                    |   |
| 100 |                                             | }                    |   |
| 101 |                                             | Spacer(modifier      | = |
| 102 | Modifier.height(8.dp))                      |                      |   |
| 103 |                                             | Text(                |   |
| 104 |                                             | text = "Tentang film |   |
| 105 | ini:",                                      |                      |   |
| 106 |                                             | style                | = |
| 107 | MaterialTheme.typography.bodyLarge.copy(    |                      |   |
| 108 |                                             | fontSize             | = |
| 109 | 14.sp,                                      |                      |   |
| 110 |                                             | color                | = |
| 111 | colorScheme.onBackground,                   |                      |   |
| 112 |                                             | fontWeight           | = |
| 113 | FontWeight.Bold                             |                      |   |
| 114 |                                             | )                    |   |
| 115 |                                             | )                    |   |
| 116 |                                             | Text(                |   |
| 117 |                                             | text                 | = |
| 118 | movie.overview,                             |                      |   |
| 119 |                                             | style                | = |
| 120 | MaterialTheme.typography.bodyMedium.copy(   |                      |   |
| 121 |                                             | fontSize             | = |
| 122 | 14.sp,                                      |                      |   |
| 123 |                                             | color                | = |
| 124 | colorScheme.onBackground                    |                      |   |
| 125 |                                             | ),                   |   |

|     |                              |                               |   |
|-----|------------------------------|-------------------------------|---|
| 115 | Modifier.padding(top = 4.dp) | modifier                      | = |
| 116 |                              | )                             |   |
| 117 |                              | }                             |   |
| 118 |                              | }                             |   |
| 119 |                              | }                             |   |
| 120 |                              | } else {                      |   |
| 121 |                              | Text(                         |   |
| 122 |                              | text = "Movie not found",     |   |
| 123 |                              | color = colorScheme.error,    |   |
| 124 |                              | modifier = Modifier           |   |
| 125 |                              | .padding(paddingValues)       |   |
| 126 |                              | .padding(16.dp)               |   |
| 127 |                              | )                             |   |
| 128 |                              | }                             |   |
| 129 |                              | }                             |   |
| 130 |                              |                               |   |
| 131 | is NetworkResult.Error -> {  |                               |   |
| 132 |                              | Text(                         |   |
| 133 |                              | text = "Error loading movie", |   |
| 134 |                              | color = colorScheme.error,    |   |
| 135 |                              | modifier = Modifier           |   |
| 136 |                              | .padding(paddingValues)       |   |
| 137 |                              | .padding(16.dp)               |   |
| 138 |                              | )                             |   |
| 139 |                              | }                             |   |
| 140 |                              | }                             |   |
| 141 |                              | }                             |   |
| 142 | }                            |                               |   |

## MainScreen.kt

Table 13 Source code MainScreen.kt

|    |                                                       |
|----|-------------------------------------------------------|
| 1  | package                                               |
|    | com.pemrogamanmobile.movielist.presentation.ui.screen |
| 2  |                                                       |
| 3  | import androidx.compose.foundation.layout.*           |
| 4  | import androidx.compose.foundation.lazy.LazyColumn    |
| 5  | import androidx.compose.foundation.lazy.items         |
| 6  | import androidx.compose.material3.*                   |
| 7  | import androidx.compose.runtime.*                     |
| 8  | import androidx.compose.ui.Alignment                  |
| 9  | import androidx.compose.ui.Modifier                   |
| 10 | import androidx.compose.ui.unit.dp                    |
| 11 | import androidx.navigation.NavController              |



```

12 import
   com.pemrogamanmobile.movielist.presentation.viewmodel.
   MovieViewModel
13 import
   com.pemrogamanmobile.movielist.utils.NetworkResult
14
15 @Composable
16 fun MainScreen(
17     viewModel: MovieViewModel,
18     navController: NavController,
19     isDarkMode: Boolean,
20     onToggleTheme: () -> Unit
21 ) {
22     val moviesResult by
   viewModel.movies.collectAsState()
23     val colorScheme = MaterialTheme.colorScheme
24
25     Scaffold(
26         topBar = {
27             TopAppBar(
28                 title = "Popular Movie",
29                 isDarkMode = isDarkMode,
30                 onToggleTheme = onToggleTheme
31             )
32         }
33     ) { paddingValues ->
34         Surface(
35             modifier = Modifier
36                 .fillMaxSize()
37                 .padding(paddingValues),
38             color = colorScheme.background
39         ) {
40             when (moviesResult) {
41                 is NetworkResult.Loading -> {
42                     Box(modifier =
   Modifier.fillMaxSize(), contentAlignment =
   Alignment.Center) {
43                         CircularProgressIndicator(color = colorScheme.primary)
44                     }
45                 }
46
47                 is NetworkResult.Success -> {
48                     val movies = (moviesResult as
   NetworkResult.Success).data
49                     LazyColumn(

```

|    |                                               |   |
|----|-----------------------------------------------|---|
| 50 | verticalArrangement                           | = |
|    | Arrangement.spacedBy(8.dp),                   |   |
| 51 | modifier                                      | = |
|    | Modifier.padding(8.dp)                        |   |
| 52 | ) {                                           |   |
| 53 | items(movies) { movie ->                      |   |
| 54 | MovieCard(movie) {                            |   |
| 55 | navController.navigate("detail/\${movie.id}") |   |
| 56 | }                                             |   |
| 57 | }                                             |   |
| 58 | }                                             |   |
| 59 | }                                             |   |
| 60 |                                               |   |
| 61 | is NetworkResult.Error -> {                   |   |
| 62 | Text(                                         |   |
| 63 | text = "Error: \${(moviesResult               |   |
|    | as NetworkResult.Error).message}",            |   |
| 64 | color = colorScheme.error,                    |   |
| 65 | modifier                                      | = |
|    | Modifier.padding(16.dp)                       |   |
| 66 | )                                             |   |
| 67 | }                                             |   |
| 68 | }                                             |   |
| 69 | }                                             |   |
| 70 | }                                             |   |
| 71 | }                                             |   |

## MovieCard.kt

Table 14 Source code MovieCard.kt

|    |                                                       |
|----|-------------------------------------------------------|
| 1  | package                                               |
|    | com.pemrogamanmobile.movielist.presentation.ui.screen |
| 2  |                                                       |
| 3  | import androidx.compose.foundation.Image              |
| 4  | import androidx.compose.foundation.layout.*           |
| 5  | import                                                |
|    | androidx.compose.foundation.shape.RoundedCornerShape  |
| 6  | import androidx.compose.material3.Button              |
| 7  | import androidx.compose.material3.ButtonDefaults      |
| 8  | import androidx.compose.material3.Card                |
| 9  | import androidx.compose.material3.CardDefaults        |
| 10 | import androidx.compose.material3.MaterialTheme       |
| 11 | import androidx.compose.material3.Text                |
| 12 | import androidx.compose.runtime.Composable            |
| 13 | import androidx.compose.ui.Alignment                  |



|    |                                    |   |
|----|------------------------------------|---|
| 58 | horizontalArrangement              | = |
|    | Arrangement.SpaceBetween,          |   |
| 59 | verticalAlignment = Alignment.Top  |   |
| 60 | ) {                                |   |
| 61 | Text(                              |   |
| 62 | text = movie.title,                |   |
| 63 | style = TextStyle(                 |   |
| 64 | fontSize = 26.sp,                  |   |
| 65 | fontWeight                         | = |
|    | FontWeight.Bold,                   |   |
| 66 | color                              | = |
|    | colorScheme.onSurface              |   |
| 67 | ),                                 |   |
| 68 | modifier = Modifier.weight(1f)     |   |
| 69 | )                                  |   |
| 70 |                                    |   |
| 71 | Spacer(modifier                    | = |
|    | Modifier.width(8.dp))              |   |
| 72 |                                    |   |
| 73 | Text(                              |   |
| 74 | text = movie.releaseDate,          |   |
| 75 | style = TextStyle(                 |   |
| 76 | fontSize = 16.sp,                  |   |
| 77 | color                              | = |
|    | colorScheme.onSurfaceVariant       |   |
| 78 | ),                                 |   |
| 79 | modifier = Modifier                |   |
| 80 | .wrapContentWidth(Alignment.End)   |   |
| 81 | .align(Alignment.CenterVertically) |   |
| 82 | )                                  |   |
| 83 | }                                  |   |
| 84 |                                    |   |
| 85 | Spacer(modifier                    | = |
|    | Modifier.height(8.dp))             |   |
| 86 |                                    |   |
| 87 | Column(modifier                    | = |
|    | Modifier.fillMaxWidth()) {         |   |
| 88 | Text(                              |   |
| 89 | text = "tentang film ini:",        |   |
| 90 | style = TextStyle(                 |   |
| 91 | fontSize = 14.sp,                  |   |
| 92 | fontWeight                         | = |
|    | FontWeight.Bold,                   |   |
| 93 | color                              | = |
|    | colorScheme.onSurface              |   |

```

94         )
95     )
96     Text(
97         text = movie.overview,
98         style = TextStyle(
99             fontSize = 14.sp,
100             color =
colorScheme.onSurface
101         ),
102         modifier = Modifier.padding(top
= 4.dp)
103     )
104 }
105
106     Spacer(modifier =
Modifier.height(12.dp))
107
108     Row(
109         modifier = Modifier.fillMaxWidth(),
110         horizontalArrangement =
Arrangement.End,
111         verticalAlignment =
Alignment.CenterVertically
112     ) {
113         Button(
114             onClick = onClick,
115             modifier = Modifier
116                 .width(100.dp)
117                 .height(44.dp),
118             colors =
ButtonDefaults.buttonColors(
119                 containerColor =
colorScheme.onSurface,
120                 contentColor =
colorScheme.onPrimary
121             )
122         ) {
123             Text("Detail", style =
TextStyle(fontSize = 16.sp))
124         }
125     }
126 }
127 }
128 }
129 }

```

**TopBarApp.kt**

Table 15 Source code TopBarApp.kt

|    |                                                         |
|----|---------------------------------------------------------|
| 1  | package                                                 |
|    | com.pemrogamanmobile.movielist.presentation.ui.screen   |
| 2  |                                                         |
| 3  | import androidx.compose.material.icons.Icons            |
| 4  | import androidx.compose.material.icons.filled.DarkMode  |
| 5  | import androidx.compose.material.icons.filled.LightMode |
| 6  | import androidx.compose.material3.*                     |
| 7  | import androidx.compose.runtime.Composable              |
| 8  |                                                         |
| 9  | @OptIn(ExperimentalMaterial3Api::class)                 |
| 10 | @Composable                                             |
| 11 | fun TopBarApp(                                          |
| 12 | title: String,                                          |
| 13 | isDarkMode: Boolean,                                    |
| 14 | onToggleTheme: () -> Unit                               |
| 15 | ) {                                                     |
| 16 | TopAppBar(                                              |
| 17 | title = {                                               |
| 18 | Text(text = title)                                      |
| 19 | },                                                      |
| 20 | actions = {                                             |
| 21 | IconButton(onClick = onToggleTheme) {                   |
| 22 | Icon(                                                   |
| 23 | imageVector = if (isDarkMode)                           |
|    | Icons.Default.LightMode else Icons.Default.DarkMode,    |
| 24 | contentDescription = "Toggle Theme"                     |
| 25 | )                                                       |
| 26 | }                                                       |
| 27 | },                                                      |
| 28 | colors = TopAppBarDefaults.topAppBarColors(             |
| 29 | containerColor =                                        |
|    | MaterialTheme.colorScheme.primary,                      |
| 30 | titleContentColor =                                     |
|    | MaterialTheme.colorScheme.onPrimary,                    |
| 31 | actionIconContentColor =                                |
|    | MaterialTheme.colorScheme.onPrimary                     |
| 32 | )                                                       |
| 33 | )                                                       |
| 34 | }                                                       |

## MainActivity.kt

Table 16 Source code MainActivity.kt

|   |                                                        |
|---|--------------------------------------------------------|
| 1 | package com.pemrogamanmobile.movielist.presentation.ui |
| 2 |                                                        |

```
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.material3.*
7 import androidx.compose.runtime.*
8 import androidx.compose.ui.graphics.Color
9 import androidx.lifecycle.ViewModelProvider
10 import androidx.navigation.NavType
11 import androidx.navigation.compose.NavHost
12 import androidx.navigation.compose.composable
13 import androidx.navigation.compose.rememberNavController
14 import androidx.navigation.navArgument
15 import com.pemrogamanmobile.movielist.data.local.db.
  MovieDatabase
16 import com.pemrogamanmobile.movielist.data.remote.
  RetrofitInstance
17 import com.pemrogamanmobile.movielist.data.repository.
  MovieRepositoryImpl
18 import com.pemrogamanmobile.movielist.domain.usecase.
  MovieUseCase
19 import
  com.pemrogamanmobile.movielist.presentation.ui.screen.
  DetailScreen
20 import
  com.pemrogamanmobile.movielist.presentation.ui.screen.
  MainScreen
21 import
  com.pemrogamanmobile.movielist.presentation.viewmodel.
  MovieViewModel
22 import
  com.pemrogamanmobile.movielist.presentation.viewmodel.
  MovieViewModelFactory
23 import
  com.pemrogamanmobile.movielist.presentation.viewmodel.
  ThemeViewModel
24 import
  com.pemrogamanmobile.movielist.presentation.viewmodel.
  ThemeViewModelFactory
25 import
  com.pemrogamanmobile.movielist.presentation.ui.theme.
  MovieListTheme
26
27 class MainActivity : ComponentActivity() {
28     override fun onCreate(savedInstanceState: Bundle?) {
29         super.onCreate(savedInstanceState)
30
```





|    |                                |               |   |   |
|----|--------------------------------|---------------|---|---|
| 63 | themeViewModel.toggleTheme() } | onToggleTheme | = | { |
| 64 |                                |               |   | ) |
| 65 |                                |               |   | } |
| 66 |                                |               |   | } |
| 67 |                                |               |   | } |
| 68 |                                |               |   | } |
| 69 |                                |               |   | } |
| 70 |                                |               |   | } |
| 71 | }                              |               |   |   |

## MovieViewModel.kt

Table 17 Source code MovieViewModel.kt

|    |                                                          |   |
|----|----------------------------------------------------------|---|
| 1  | package                                                  |   |
| 2  | com.pemrogamanmobile.movielist.presentation.viewmodel    |   |
| 3  | import androidx.lifecycle.ViewModel                      |   |
| 4  | import androidx.lifecycle.viewModelScope                 |   |
| 5  | import com.pemrogamanmobile.movielist.domain.model.Movie |   |
| 6  | import com.pemrogamanmobile.movielist.domain.usecase.    |   |
| 7  | MovieUseCase                                             |   |
| 8  | import                                                   |   |
| 9  | com.pemrogamanmobile.movielist.utils.NetworkResult       |   |
| 10 | import kotlinx.coroutines.flow.MutableStateFlow          |   |
| 11 | import kotlinx.coroutines.flow.StateFlow                 |   |
| 12 | import kotlinx.coroutines.launch                         |   |
| 13 | class MovieViewModel(private val                         |   |
| 14 | movieUseCase: MovieUseCase) : ViewModel() {              |   |
| 15 | private val _movies                                      | = |
| 16 | MutableStateFlow<NetworkResult<List<Movie>>>             |   |
| 17 | (NetworkResult.Loading)                                  |   |
| 18 | val movies: StateFlow<NetworkResult<List<Movie>>>        |   |
| 19 | get() = _movies                                          |   |
| 20 | init {                                                   |   |
| 21 | fetchMovies()                                            |   |
| 22 | }                                                        |   |
| 23 | fun fetchMovies() {                                      |   |
| 24 | viewModelScope.launch {                                  |   |
| 25 | movieUseCase.getPopularMovies().collect {                |   |
| 26 | _movies.value = it                                       |   |
| 27 | }                                                        |   |
| 28 | }                                                        |   |

|    |   |
|----|---|
| 26 | } |
| 27 | } |

## MovieViewModelFactory.kt

Table 18 Source code MovieViewModelFactory.kt

|    |                                                         |
|----|---------------------------------------------------------|
| 1  | package                                                 |
|    | com.pemrogamanmobile.movielist.presentation.viewmodel   |
| 2  |                                                         |
| 3  | import androidx.lifecycle.ViewModel                     |
| 4  | import androidx.lifecycle.ViewModelProvider             |
| 5  | import                                                  |
|    | com.pemrogamanmobile.movielist.domain.usecase.MovieUse  |
|    | Case                                                    |
| 6  | class MovieViewModelFactory(private val useCase:        |
| 7  | MovieUseCase) : ViewModelProvider.Factory {             |
|    | override fun <T : ViewModel> create(modelClass:         |
| 8  | Class<T>): T {                                          |
|    | if                                                      |
| 9  | (modelClass.isAssignableFrom(MovieViewModel::class.java |
|    | a)) {                                                   |
|    | return MovieViewModel(useCase) as T                     |
| 10 | }                                                       |
| 11 | throw IllegalArgumentException("Unknown                 |
| 12 | ViewModel class")                                       |
|    | }                                                       |
| 13 | }                                                       |
| 14 |                                                         |

## ThemeViewModel.kt

Table 19 Source code ThemeViewModel.kt

|   |                                                         |
|---|---------------------------------------------------------|
| 1 | package                                                 |
|   | com.pemrogamanmobile.movielist.presentation.viewmodel   |
| 2 |                                                         |
| 3 | import androidx.lifecycle.ViewModel                     |
| 4 | import androidx.lifecycle.viewModelScope                |
| 5 | import com.pemrogamanmobile.movielist.data.preferences. |
|   | PreferencesManager                                      |
| 6 | import kotlinx.coroutines.flow.MutableStateFlow         |
| 7 | import kotlinx.coroutines.flow.StateFlow                |
| 8 | import kotlinx.coroutines.launch                        |
| 9 |                                                         |

|    |                                                                                          |
|----|------------------------------------------------------------------------------------------|
| 10 | class ThemeViewModel(private val preferencesManager: PreferencesManager) : ViewModel() { |
| 11 |                                                                                          |
| 12 | private val _isDarkMode =                                                                |
|    | MutableStateFlow(preferencesManager.isDarkMode())                                        |
| 13 | val isDarkMode: StateFlow<Boolean> = _isDarkMode                                         |
| 14 |                                                                                          |
| 15 | fun toggleTheme() {                                                                      |
| 16 | val newMode = !_isDarkMode.value                                                         |
| 17 | _isDarkMode.value = newMode                                                              |
| 18 | viewModelScope.launch {                                                                  |
| 19 | preferencesManager.saveThemeSetting(newMode)                                             |
| 20 | }                                                                                        |
| 21 | }                                                                                        |
| 22 | }                                                                                        |

## ThemeViewModelFactory.kt

Table 20 Source code ThemeViewModelFactory.kt

|    |                                                                                         |
|----|-----------------------------------------------------------------------------------------|
| 1  | package                                                                                 |
|    | com.pemrogamanmobile.movielist.presentation.viewmodel                                   |
| 2  |                                                                                         |
| 3  | import android.content.Context                                                          |
| 4  | import androidx.lifecycle.ViewModel                                                     |
| 5  | import androidx.lifecycle.ViewModelProvider                                             |
| 6  | import com.pemrogamanmobile.movielist.data.preferences.                                 |
|    | PreferencesManager                                                                      |
| 7  |                                                                                         |
| 8  | class ThemeViewModelFactory(private val context: Context) : ViewModelProvider.Factory { |
| 9  | override fun <T : ViewModel> create(modelClass: Class<T>): T {                          |
| 10 | val prefs = PreferencesManager(context)                                                 |
| 11 | return ThemeViewModel(prefs) as T                                                       |
| 12 | }                                                                                       |
| 13 | }                                                                                       |

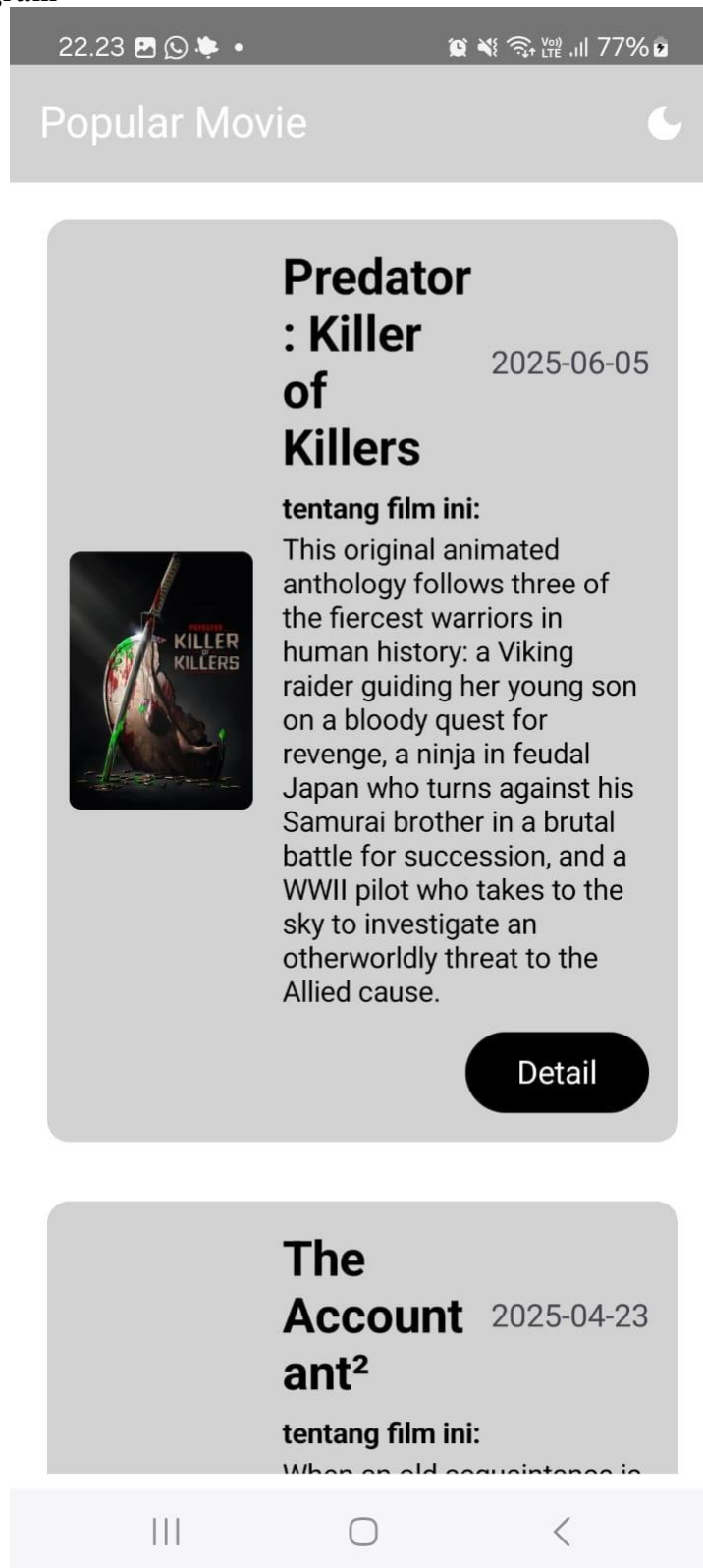
## NetworkResult.kt

Table 21 Source code NetworkResult.kt

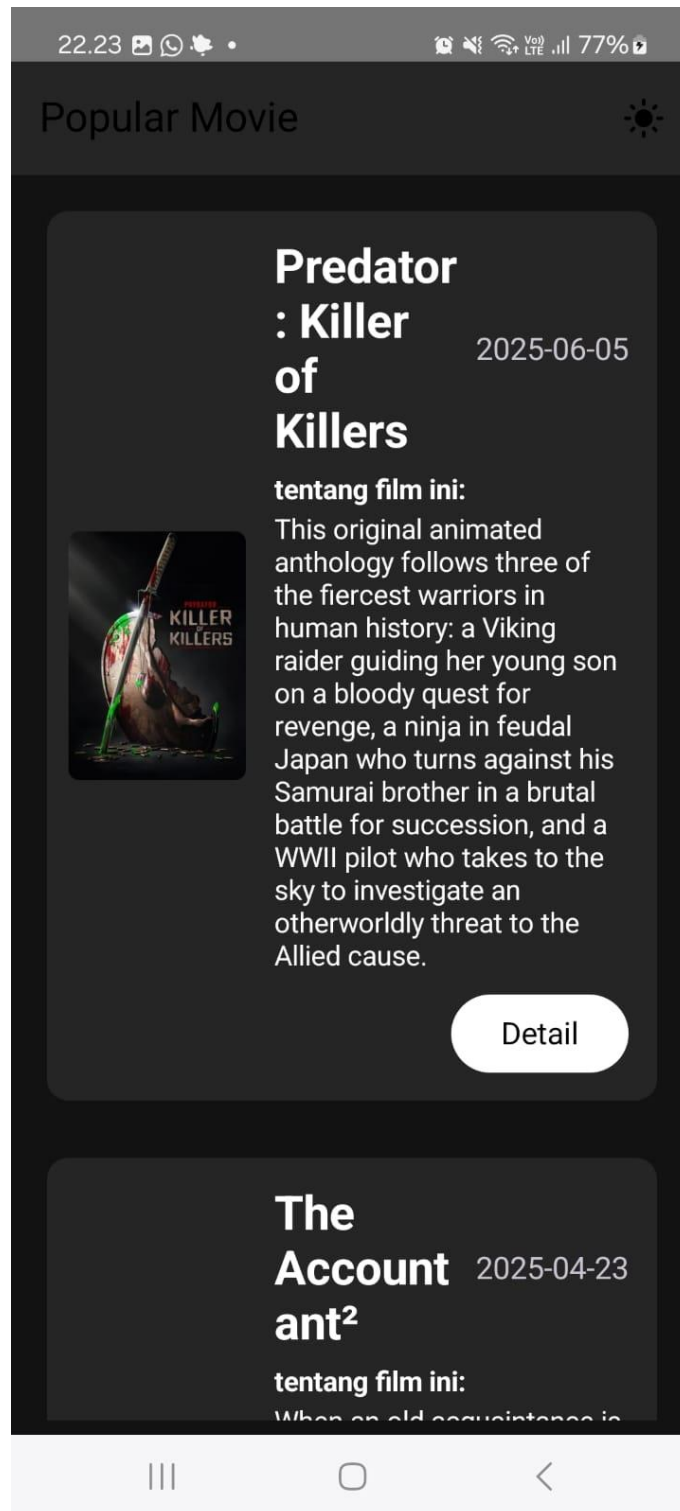
|   |                                              |
|---|----------------------------------------------|
| 1 | package com.pemrogamanmobile.movielist.utils |
| 2 |                                              |
| 3 | sealed class NetworkResult<out T> {          |

|   |                                          |
|---|------------------------------------------|
| 4 | data class Success<out T>(val data: T):  |
|   | NetworkResult<T>()                       |
| 5 | data class Error(val message: String):   |
|   | NetworkResult<Nothing>()                 |
| 6 | object Loading: NetworkResult<Nothing>() |
| 7 | }                                        |

## B. Output Program



Gambar 1 Screenshot output light mode



Gambar 2 Screenshot output dark mode

### **C. Pembahasan**

#### **COMPOSE :**

##### **MovieDao.kt**

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.data.local.dao` yang berfungsi sebagai nama paket dari modul DAO dan tempat menyimpan file ini dalam struktur aplikasi.

Pada baris ke [3] sampai [7], dilakukan impor berbagai komponen penting dari Android Room, yaitu `Dao`, `Insert`, `OnConflictStrategy`, dan `Query`, serta entitas `MovieEntity` yang merepresentasikan tabel `movies` di database lokal.

Pada baris ke [9] terdapat anotasi `@Dao` yang menandai interface ini sebagai `Data Access Object` untuk Room, yakni komponen yang bertanggung jawab atas operasi baca-tulis ke database.

Pada baris ke [10] hingga [19] dideklarasikan interface `MovieDao` yang berisi tiga method suspend untuk melakukan operasi database secara asynchronous.

Pada baris ke [11] anotasi `@Insert(onConflict = OnConflictStrategy.REPLACE)` menyatakan bahwa saat menyisipkan daftar entitas, jika terjadi konflik (misalnya `primary key` duplikat), data lama akan digantikan dengan data baru. Baris ke [12] suspend fun `insertMovies(movies: List<MovieEntity>)` adalah method untuk menyimpan kumpulan `MovieEntity`.

Pada baris ke [14] anotasi `@Query("SELECT * FROM movies")` mendefinisikan perintah SQL untuk mengambil semua baris dari tabel `movies`. Baris ke [15] suspend fun `getAllMovies(): List<MovieEntity>` mengembalikan daftar seluruh film yang tersimpan.

Pada baris ke [17] anotasi `@Query("DELETE FROM movies")` mendefinisikan perintah SQL untuk menghapus semua data dalam tabel `movies`. Baris ke [18] suspend fun `clearMovies()` mengeksekusi operasi pembersihan tersebut.

##### **MovieDatabase.kt:**

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.data.local.db` yang berfungsi sebagai nama paket dari modul database lokal, tempat menyimpan kelas konfigurasi Room Database.

Pada baris ke [3] sampai [6], dilakukan impor `Context` dari Android serta komponen Room yaitu `Database`, `Room`, dan `RoomDatabase`, yang diperlukan untuk mendefinisikan dan membangun database. Pada baris ke [7] dan [8], diimpor `MovieDao` dan `MovieEntity`—DAO dan entitas yang akan dikelola oleh database ini.

Pada baris ke [10] terdapat anotasi `@Database(entities = [MovieEntity::class], version = 1, exportSchema = false)`, yang menyatakan bahwa kelas berikutnya adalah Room Database dengan tabel berdasarkan `MovieEntity`, menggunakan skema versi 1 dan tanpa mengeksport skema ke file.

Pada baris ke [11] dideklarasikan `abstract class MovieDatabase : RoomDatabase()` sebagai subclass `RoomDatabase` yang akan menjadi akses utama ke database.

Pada baris ke [12] terdapat `abstract fun movieDao(): MovieDao` yang mendefinisikan fungsi abstrak untuk mendapatkan instance `MovieDao`, memungkinkan aplikasi melakukan operasi CRUD pada tabel `movies`.

Pada baris ke [14] hingga [26] terdapat companion object yang menyiapkan pola singleton untuk database agar hanya ada satu instance selama siklus hidup aplikasi.

Pada baris ke [15] anotasi `@Volatile` pada `private var INSTANCE: MovieDatabase? = null` memastikan bahwa setiap thread akan melihat pembaruan terakhir pada variabel ini.

Pada baris ke [18] fungsi `fun getDatabase(context: Context): MovieDatabase` menyediakan akses publik ke instance database.

Pada baris ke [19] digunakan pengecekan `INSTANCE ?`: untuk mengembalikan instance jika sudah ada, atau masuk ke blok `synchronized(this)` agar pembuatan instance hanya terjadi sekali secara thread-safe.

Pada baris ke [19] sampai akhir, `Room.databaseBuilder` membangun database dengan `context.applicationContext`, kelas `MovieDatabase::class.java`, dan nama file `"movie_db"`. Metode `fallbackToDestructiveMigration()` memastikan bahwa saat versi skema berubah tanpa migrasi yang ditentukan, database lama akan dihapus dan dibuat ulang. Hasil builder kemudian dibangun dengan `build()` dan langsung di-assign ke `INSTANCE` lewat `also { INSTANCE = it }`.

### **MovieEntity.kt**

Pada baris ke [1] terdapat `package com.pemrogamanmobile.movielist.data.local.entity` yang berfungsi sebagai nama paket untuk entitas database lokal dan tempat menyimpan file ini dalam struktur aplikasi.

Pada baris ke [3] sampai [4], dilakukan impor `Entity` dan `PrimaryKey` dari `Android Room`, yang digunakan untuk menandai kelas ini sebagai tabel dan menentukan kolom kunci utama.

Pada baris ke [6] terdapat anotasi `@Entity(tableName = "movies")` yang menyatakan bahwa `MovieEntity` akan dipetakan ke tabel `SQLite` bernama `movies`.

Pada baris ke [7] sampai [13] dideklarasikan `data class MovieEntity(...)` yang merepresentasikan struktur baris pada tabel `movies` dengan lima properti:



Pada baris ke [8], anotasi `@PrimaryKey` pada properti `val id: Int` menandai `id` sebagai kunci utama tabel, unik untuk setiap film.

Pada baris ke [9], `val title: String` menyimpan judul film.

Pada baris ke [10], `val overview: String` menyimpan ringkasan atau sinopsis film.

Pada baris ke [11], `val posterPath: String` menyimpan path atau URL gambar poster film.

Pada baris ke [12], `val releaseDate: String` menyimpan tanggal perilis film dalam format string.

### **PreferencesManager.kt**

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.data.preferences` yang berfungsi sebagai nama paket untuk modul pengelolaan preferensi aplikasi dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [4], dilakukan impor `Context` dan `SharedPreferences` dari `Android`, yang dibutuhkan untuk mengakses dan menyimpan data preferensi pengguna.

Pada baris ke [6] dideklarasikan class `PreferencesManager(context: Context)`, sebuah kelas yang menerima `Context` saat instansiasi untuk menginisialisasi akses ke `SharedPreferences`.

Pada baris ke [7] terdapat `private val prefs: SharedPreferences = context.getSharedPreferences("movie_prefs", Context.MODE_PRIVATE)` yang membuat atau membuka file preferensi bernama `"movie_prefs"` dengan mode private, sehingga hanya aplikasi ini yang dapat membacanya.

Pada baris ke [9] sampai [11] didefinisikan fun `saveThemeSetting(isDarkMode: Boolean)`, sebuah fungsi yang menyimpan pilihan tema (gelap atau terang) dengan mengedit `prefs`, menempatkan boolean di bawah kunci `"dark_mode"`, lalu memanggil `apply()` agar perubahan langsung ditulis.

Pada baris ke [13] terdapat fun `isDarkMode(): Boolean = prefs.getBoolean("dark_mode", false)` yang mengembalikan nilai boolean dari preferensi `"dark_mode"`, dengan default `false` jika belum pernah disimpan sebelumnya.

### **ApiService.kt**

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.data.remote.api` yang berfungsi sebagai nama paket untuk modul akses API remote dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [5], dilakukan impor `MovieResponseDto` dari paket `DTO`, serta anotasi `@GET` dan `@Query` dari `Retrofit`, yang dibutuhkan untuk mendefinisikan endpoint HTTP dan parameter query.

Pada baris ke [7] dideklarasikan interface ApiService sebagai kontrak yang menjelaskan bagaimana aplikasi akan berkomunikasi dengan API remote menggunakan Retrofit.

Pada baris ke [7] sampai [12] didefinisikan fungsi getPopularMovies:

Pada baris ke [8] anotasi @GET("movie/popular") menyatakan bahwa pemanggilan ini akan melakukan HTTP GET ke path movie/popular.

Pada baris ke [9] suspend fun getPopularMovies( menandai method ini sebagai suspend, sehingga dapat dipanggil dari coroutine untuk operasi non-blocking.

Pada baris ke [10] anotasi @Query("api\_key") apiKey: String menetapkan bahwa parameter apiKey akan diteruskan sebagai query string ?api\_key=... pada request.

Pada baris ke [11] ): MovieResponseDto menunjukkan bahwa respons JSON dari endpoint akan dipetakan menjadi objek MovieResponseDto.

#### **MovieDto.kt:**

Pada baris ke [1] terdapat package com.pemrogramanmobile.movielist.data.remote.dto yang berfungsi sebagai nama paket untuk modul DTO (Data Transfer Object) yang memetakan data JSON dari API ke objek Kotlin.

Pada baris ke [3] sampai [4], dilakukan impor Serializable dan Serializable dari kotlinx.serialization, yang memungkinkan kelas ini diserialisasi dan mendeserialisasi JSON secara otomatis.

Pada baris ke [6] terdapat anotasi @Serializable yang menandai data class MovieDto sebagai objek yang dapat di-serialize dan di-deserialize oleh pustaka kotlinx.serialization.

Pada baris ke [7] sampai [13] dideklarasikan data class MovieDto(...) dengan lima properti yang memetakan satu film dari respons API:

val id: Int (baris ke [8]) menyimpan ID unik film.

val title: String (baris ke [9]) menyimpan judul film.

val overview: String (baris ke [10]) menyimpan sinopsis film.

@SerializedName("poster\_path") val posterPath: String (baris ke [11]) memetakan kunci JSON "poster\_path" ke properti posterPath.

@SerializedName("release\_date") val releaseDate: String (baris ke [12]) memetakan kunci JSON "release\_date" ke properti releaseDate.

Pada baris ke [15] kembali terdapat anotasi `@Serializable` yang menandai data class `MovieResponseDto` dapat di-serialize.

Pada baris ke [16] sampai [18] dideklarasikan data class `MovieResponseDto(val results: List<MovieDto>)` yang merepresentasikan struktur JSON respons top-level, dengan `results` sebagai daftar objek `MovieDto`.

#### **RetrofitInstance.kt**

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movies.data.remote` yang berfungsi sebagai nama paket untuk modul komunikasi jaringan aplikasi dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [7], dilakukan impor berbagai komponen penting untuk membangun klien Retrofit dengan dukungan serialisasi JSON: `asConverterFactory` dari Jake Wharton untuk menghubungkan `kotlinx.serialization` ke Retrofit, `ApiService` sebagai interface endpoint, `Json` dari `kotlinx.serialization` untuk konfigurasi parser, ekstensi `toMediaType` dari `OkHttp` untuk mendefinisikan media type, dan Retrofit sebagai inti HTTP client.

Pada baris ke [9] dideklarasikan object `RetrofitInstance`, sebuah singleton yang menyiapkan dan menyediakan klien Retrofit serta service API-nya.

Pada baris ke [10] terdapat `private const val BASE_URL = "https://api.themoviedb.org/3/"` yang menyimpan URL dasar untuk semua permintaan ke The Movie Database API.

Pada baris ke [12] sampai [17] didefinisikan `private val retrofit by lazy { ... }` sehingga instance Retrofit hanya dibuat sekali secara thread-safe ketika pertama kali diakses. Di dalam blok `lazy`: Dipanggil `Retrofit.Builder()` untuk memulai konstruksi klien, `baseUrl(BASE_URL)` menetapkan alamat dasar API, `addConverterFactory(...)` menambahkan converter JSON yang dikonfigurasi dengan `Json { ignoreUnknownKeys = true }` agar parser mengabaikan field tak dikenal, lalu diubah menjadi `ConverterFactory` dengan media type `"application/json"`, `build()` memfinalisasi dan menghasilkan instance Retrofit.

Pada baris ke [19] sampai [21] terdapat `val api: ApiService by lazy { retrofit.create(ApiService::class.java) }` yang menyediakan satu-satunya instance `ApiService`, siap untuk dipanggil dari coroutine di seluruh aplikasi.

#### **MovieRepositoryImpl.kt**

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movies.data.repository` yang berfungsi sebagai nama paket untuk modul repositori aplikasi dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [9], dilakukan impor `MovieDao` dan `ApiService` sebagai komponen utama untuk akses data lokal dan remote, `Movie` sebagai model domain,

MovieRepository sebagai kontrak repositori, NetworkResult untuk membungkus status jaringan, serta Flow dan flow dari Coroutines untuk aliran data asinkron.

Pada baris ke [11] dideklarasikan class MovieRepositoryImpl(private val api: ApiService, private val dao: MovieDao) : MovieRepository yang mengimplementasikan kontrak MovieRepository, menerima instance ApiService dan MovieDao lewat konstruktor untuk melakukan operasi data.

Pada baris ke [15] hingga [43] diimplementasikan fungsi override fun getPopularMovies(): Flow<NetworkResult<List<Movie>>> = flow { ... } yang mengembalikan sebuah Flow berisi NetworkResult dari daftar Movie. Di dalam blok flow:

Pada baris ke [16] emit(NetworkResult.Loading) mengirim status loading sebelum operasi dimulai.

Pada baris ke [17] sampai [33] dibungkus dalam try { ... }, di mana pertama dipanggil api.getPopularMovies(...) untuk mengambil data film populer, lalu hasil response.results dipetakan menjadi model domain Movie. Selanjutnya, dao.clearMovies() menghapus cache lama, diikuti dao.insertMovies(...) yang menyimpan daftar film baru sebagai MovieEntity. Setelah itu, emit(NetworkResult.Success(movies)) mengirim status sukses beserta data.

Pada baris ke [33] sampai [42] terdapat blok catch(e: Exception) { ... } yang menangani kegagalan jaringan. Di sini, data cache diambil lewat dao.getAllMovies(), dipetakan menjadi model Movie, dan jika tidak kosong maka emit(NetworkResult.Success(cached)) mengirim data cache; jika kosong, dikembalikan emit(NetworkResult.Error(e.message ?: "Unknown Error")) sebagai status error.

Penjelasan caching strategy:

Caching strategy yang digunakan adalah NetworkBoundResource versi sederhana, Dimana aplikasi pertama kali mencoba untuk mengambil data lewat api, kemudian menyimpannya di database local menggunakan room. Jika data berhasil diambil, maka akan langsung ditampilkan. Sedangkan jika terjadi error maka mengambil data dari database local dan ditampilkan. Strategi ini dipilih karena paling sederhana dan cukup efektif untuk membuat aplikasi dapat berjalan secara offline.

## **Movie.kt**

Pada baris ke [1] terdapat package com.pemrogramanmobile.movielist.domain.model yang berfungsi sebagai nama paket untuk modul model domain aplikasi, tempat menyimpan definisi kelas yang merepresentasikan data bisnis.

Pada baris ke [3] terdapat deklarasi data class Movie( yang mendefinisikan kelas data Movie—sebuah model domain immutable dengan otomatisasi fungsi equals(), hashCode(), toString(), dan copy().

Pada baris ke [4] val id: Int menyatakan properti id bertipe Int yang berfungsi sebagai pengenalan unik (identifier) untuk setiap objek Movie.

Pada baris ke [5] `val title: String` mendeklarasikan properti `title` bertipe `String` untuk menyimpan judul film.

Pada baris ke [6] `val overview: String` mendefinisikan properti `overview` bertipe `String` yang berisi sinopsis atau ringkasan film.

Pada baris ke [7] `val posterPath: String` menyatakan properti `posterPath` bertipe `String` yang menyimpan path atau URL ke gambar poster film.

Pada baris ke [8] `val releaseDate: String` mendeklarasikan properti `releaseDate` bertipe `String` yang menyimpan tanggal rilis film dalam format teks.

### **MovieRepository.kt**

Pada baris ke [1] terdapat `package com.pemrogamanmobile.movies.domain.repository` yang berfungsi sebagai nama paket untuk modul repositori domain aplikasi dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [5], dilakukan impor `Movie` dari paket `domain.model` yang merepresentasikan data film di lapisan domain, `NetworkResult` dari utilitas yang membungkus status jaringan (`Loading`, `Success`, `Error`), serta `Flow` dari `Coroutines` untuk menyediakan aliran data yang bersifat `asynchronous`.

Pada baris ke [7] terdapat interface `MovieRepository` yang mendefinisikan kontrak repositori untuk operasi terkait entitas `Movie`, memisahkan logika pengambilan data dari implementasi spesifik.

Pada baris ke [8] dideklarasikan `fun getPopularMovies(): Flow<NetworkResult<List<Movie>>>` sebagai satu-satunya method dalam interface ini, yang mengembalikan sebuah `Flow` berisi `NetworkResult` dari daftar model `Movie`, memungkinkan konsumen menerima status `loading`, hasil sukses, atau error secara reaktif.

Pada baris ke [9] terdapat penutup kurung kurawal `}` yang mengakhiri blok deklarasi interface `MovieRepository`.

### **MovieUseCase.kt**

Pada baris ke [1] terdapat `package com.pemrogamanmobile.movies.domain.usecase` yang berfungsi sebagai nama paket untuk modul use case di lapisan domain aplikasi dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [6], dilakukan impor `Movie` dari paket `domain.model` untuk representasi data film di domain, `MovieRepository` dari paket `domain.repository` sebagai kontrak penyedia data, `NetworkResult` dari utilitas untuk membungkus status jaringan, serta `Flow` dari `Coroutines` untuk menyediakan aliran data yang bersifat `asynchronous`.

Pada baris ke [8] dideklarasikan class `MovieUseCase`(private val repository: `MovieRepository`) yang merupakan kelas use case dengan satu dependensi `MovieRepository`, bertugas mengeksekusi logika bisnis yang terkait dengan pengambilan film populer.

Pada baris ke [9] sampai [12] terdapat fungsi fun `getPopularMovies(): Flow<NetworkResult<List<Movie>>>` { return repository.getPopularMovies() } yang memanggil langsung method `getPopularMovies` dari repository dan mengembalikan Flow berisi `NetworkResult` daftar `Movie`, menghubungkan lapisan domain dengan repository tanpa menambah logika tambahan.

### **DetailScreen.kt**

Pada baris ke [1] terdapat deklarasi package `com.pemrogamanmobile.movielist.presentation.ui.screen` yang menunjukkan bahwa file ini berada dalam struktur folder tersebut dan merupakan bagian dari modul `presentation.ui.screen` dalam aplikasi.

Pada baris ke [3] sampai [20], dilakukan impor berbagai komponen dari Jetpack Compose dan pustaka lain seperti `Image`, `Column`, `Row`, `Scaffold`, `Surface`, `Text`, serta fungsi `rememberScrollState`, `clip`, dan `ContentScale`. Selain itu, komponen tambahan dari Coil yaitu `rememberAsyncImagePainter` digunakan untuk menampilkan gambar dari URL secara asinkron, serta `ViewModel` dan utilitas `NetworkResult` untuk mengelola data film.

Pada baris ke [22] hingga [28], didefinisikan fungsi composable `DetailScreen` yang menerima empat parameter: `viewModel` dari tipe `MovieViewModel`, `movieId` sebagai ID film yang akan ditampilkan, `isDarkMode` sebagai indikator mode gelap atau terang, dan `onToggleTheme` sebagai lambda untuk mengubah tema.

Pada baris ke [29], dibuat variabel `moviesResult` yang akan memantau perubahan data dari `viewModel.movies` secara reaktif menggunakan `collectAsState`.

Pada baris ke [30], `scrollState` dideklarasikan menggunakan `rememberScrollState()` agar tampilan dapat discroll secara vertikal jika kontennya melebihi ukuran layar.

Pada baris ke [31], `colorScheme` didefinisikan untuk mengambil skema warna aktif dari `MaterialTheme`, baik pada mode terang maupun gelap.

Pada baris ke [33] hingga [41], digunakan `Scaffold` untuk membuat struktur tata letak utama aplikasi yang mencakup `topBar` dan isi konten. Bagian `topBar` menampilkan composable `TopBarApp` dengan judul "Detail Movie" serta parameter untuk menangani tema gelap.

Pada baris ke [48] hingga [50], ketika `moviesResult` berada pada status `Loading`, maka ditampilkan `CircularProgressIndicator` di tengah layar sebagai indikator proses pemuatan data film.

Pada baris ke [53] hingga [129], jika `moviesResult` merupakan `NetworkResult.Success`, maka data film dicari berdasarkan ID menggunakan method `find`. Jika film ditemukan, maka

akan ditampilkan informasi lengkapnya di dalam Surface yang membungkus tampilan dengan warna latar dari tema aktif.

Pada baris ke [56] hingga [61], digunakan Column yang dibungkus `verticalScroll(scrollState)` agar seluruh konten dapat discroll. `Padding 16.dp` ditambahkan untuk memberi jarak dari tepi layar.

Pada baris ke [67] hingga [75], ditampilkan gambar poster film menggunakan Image dengan painter dari URL TMDb dan `contentDescription` dari judul film. Gambar diberi tinggi `400.dp`, lebar penuh, dan bentuk sudut membulat menggunakan `clip(RoundedCornerShape(6.dp))`.

Pada baris ke [77] hingga [117], Column digunakan kembali untuk menampung informasi detail film seperti judul, tanggal rilis, label deskripsi, dan sinopsis.

Pada baris ke [78] hingga [96], Row digunakan untuk menyusun judul film dan tanggal rilis secara horizontal dengan pengaturan `Arrangement.SpaceBetween`. Judul film menggunakan gaya `headlineSmall` dan tanggal rilis menggunakan `bodyMedium` dengan warna teks transparan 60%.

Pada baris ke [98], `Spacer` ditambahkan setinggi `8.dp` sebagai pemisah antar elemen.

Pada baris ke [100] hingga [107], ditampilkan teks "Tentang film ini:" sebagai label menggunakan gaya `bodyLarge`, ukuran font `14.sp`, warna `onBackground`, dan `FontWeight.Bold`.

Pada baris ke [109] hingga [116], ditampilkan teks deskripsi film yang diambil dari `movie.overview` menggunakan gaya `bodyMedium` dengan ukuran font `14.sp`, warna `onBackground`, dan jarak atas `4.dp` dari label sebelumnya.

Pada baris ke [120] hingga [127], jika film dengan ID tersebut tidak ditemukan dalam data sukses, maka akan ditampilkan teks "Movie not found" dengan warna error dari tema dan `padding 16.dp`.

Pada baris ke [131] hingga [139], jika `moviesResult` merupakan `NetworkResult.Error`, maka akan ditampilkan teks "Error loading movie" dengan warna error dan padding yang sama seperti kondisi film tidak ditemukan.

## **MainScreen.kt**

Pada baris ke [1] terdapat deklarasi `package com.pemrogramanmobile.movielist.presentation.ui.screen` yang menunjukkan bahwa file ini berada dalam struktur paket bernama `presentation.ui.screen` di dalam modul `movielist` dari aplikasi. Package ini digunakan untuk mengelompokkan dan mengorganisasi kode agar lebih rapi.

Pada baris ke [3] sampai [13], dilakukan impor berbagai komponen dari Android Jetpack Compose seperti `layout`, `lazy column`, `material3`, serta elemen-elemen penting seperti

Modifier, Alignment, dan dp (density-independent pixels). Selain itu, juga diimpor NavController dari androidx.navigation untuk navigasi antar layar, serta MovieViewModel sebagai ViewModel yang menyimpan dan mengelola data film, dan NetworkResult yang digunakan untuk merepresentasikan status permintaan data (loading, sukses, atau error).

Pada baris ke [15] sampai [21], terdapat fungsi composable bernama mainScreen yang menerima empat parameter: viewModel dari tipe MovieViewModel untuk mengakses data film, NavController dari tipe NavController untuk navigasi antar layar, isDarkMode bertipe Boolean untuk menentukan apakah tema gelap sedang aktif, dan onToggleTheme, yaitu lambda function yang dipanggil saat pengguna ingin mengganti tema aplikasi.

Pada baris ke [22] sampai [23], dideklarasikan sebuah variabel moviesResult menggunakan delegasi by dari collectAsState() untuk memantau perubahan aliran data film dari viewModel.movies. Nilai ini akan otomatis diperbarui saat data dalam flow berubah. Selain itu, colorScheme diambil dari MaterialTheme.colorScheme untuk mendapatkan skema warna saat ini (termasuk tema terang atau gelap).

Pada baris ke [25] sampai [33], digunakan komponen Scaffold untuk membuat struktur layar standar dengan top bar dan konten utama. Di dalam topBar, fungsi TopAppBar dipanggil dengan parameter title, isDarkMode, dan onToggleTheme untuk menampilkan judul halaman dan menyediakan tombol pengganti tema.

Pada baris ke [34] hingga [39], elemen Surface digunakan untuk membungkus isi tampilan utama layar. Modifier digunakan untuk mengisi seluruh ukuran layar dan memberi padding sesuai dengan paddingValues dari Scaffold. Warna latar belakang diatur menggunakan colorScheme.background untuk menyesuaikan dengan tema saat ini.

Pada baris ke [40] hingga [45], terdapat pengecekan kondisi status moviesResult menggunakan when. Jika statusnya adalah Loading, maka akan ditampilkan CircularProgressIndicator yang ditempatkan di tengah layar untuk menandakan bahwa data sedang dimuat.

Pada baris ke [47] hingga [59], jika statusnya adalah Success, maka data film akan diambil dari moviesResult yang dikonversi menjadi tipe NetworkResult.Success, kemudian hasilnya ditampilkan dalam LazyColumn. Setiap item dalam daftar akan digambarkan menggunakan MovieCard, dan ketika sebuah kartu ditekan, maka akan menavigasi ke halaman detail film berdasarkan movie.id.

Pada baris ke [61] hingga [67], jika statusnya adalah Error, maka akan ditampilkan teks dengan pesan error yang diambil dari moviesResult sebagai NetworkResult.Error. Teks ini diberi warna colorScheme.error dan padding sebesar 16.dp agar tidak menempel di tepi layar.

### **MovieCard.kt**

Pada baris ke [1] terdapat deklarasi package com.pemrogramanmobile.movielist.presentation.ui.screen yang berfungsi sebagai penanda lokasi file ini dalam struktur paket proyek Android, serta membantu dalam pengorganisasian dan pemanggilan kelas atau fungsi dari bagian lain aplikasi.



Pada baris ke [3] sampai [22], dilakukan impor berbagai komponen dari Jetpack Compose seperti Image, Row, Column, Spacer, Card, Text, dan lain-lain, yang digunakan untuk membangun antarmuka pengguna. Selain itu, digunakan juga komponen tema dari MaterialTheme, Button, RoundedCornerShape, serta ContentScale dan FontWeight. Library Coil juga diimpor untuk memuat gambar secara asynchronous melalui rememberAsyncImagePainter. Impor lainnya mencakup Movie, yaitu model data domain untuk informasi film.

Pada baris ke [24] sampai [129], terdapat fungsi composable bernama MovieCard yang menerima dua parameter, yaitu movie bertipe Movie untuk menampilkan data film, dan onClick sebagai fungsi lambda yang dijalankan saat tombol “Detail” ditekan.

Pada baris ke [26], dideklarasikan val colorScheme = MaterialTheme.colorScheme untuk mengambil skema warna saat ini dari tema Material3, agar elemen UI dapat mengikuti warna yang konsisten dalam aplikasi.

Pada baris ke [28] hingga [35], digunakan komponen Card dari Material3 sebagai wadah utama tampilan kartu film. Kartu ini dibuat memenuhi lebar penuh dengan padding 12.dp, serta diberi warna latar belakang sesuai warna utama dari tema menggunakan CardDefaults.cardColors.

Pada baris ke [36], digunakan Row agar isi dari kartu tersusun secara horizontal, sehingga gambar poster film dan informasi lainnya dapat ditampilkan sejajar dari kiri ke kanan.

Pada baris ke [38] sampai [48], digunakan Image untuk menampilkan poster film yang diambil dari URL menggunakan path movie.posterPath dan dimuat menggunakan rememberAsyncImagePainter. Gambar diberi padding kiri 12.dp, lebar 100.dp, tinggi 140.dp, serta dipotong menggunakan clip(RoundedCornerShape(6.dp)) agar sudutnya membulat. Posisi gambar disejajarkan ke tengah secara vertikal dan menggunakan ContentScale.Crop agar konten gambar menyesuaikan kotak.

Pada baris ke [50] hingga [126], digunakan Column di sebelah kanan gambar sebagai kolom utama yang memuat informasi tentang film. Kolom ini diberi padding 16.dp dan mengisi lebar penuh agar semua elemen teks dan tombol dapat ditata secara vertikal.

Pada baris ke [56] hingga [83], digunakan Row untuk menampilkan judul film dan tanggal rilis secara berdampingan di bagian atas kolom. Judul film ditampilkan menggunakan Text dengan ukuran font 26.sp, tebal (FontWeight.Bold), dan warna teks menyesuaikan colorScheme.onSurface. Teks judul juga diberikan Modifier.weight(1f) agar bisa mengisi ruang tersisa. Di sampingnya, terdapat teks releaseDate yang ditampilkan dengan ukuran font 16.sp dan warna teks onSurfaceVariant.

Pada baris ke [85], diberikan Spacer dengan tinggi 8.dp untuk memberi jarak antara bagian judul dan deskripsi.

Pada baris ke [87] hingga [104], digunakan Column untuk menampilkan teks label “tentang film ini:” dan overview dari film. Label tersebut menggunakan font tebal dan ukuran kecil (14.sp), dan deskripsi film ditampilkan di bawahnya dengan warna teks yang sama tapi tidak ditebalkan, serta diberi padding atas 4.dp untuk memisahkannya dari label.

Pada baris ke [106], diberikan lagi Spacer dengan tinggi 12.dp untuk memberi jarak antara bagian deskripsi dan tombol.

Pada baris ke [108] hingga [125], digunakan Row untuk menempatkan tombol “Detail” di bagian kanan bawah kartu. Row ini mengatur agar tombol disejajarkan ke kanan menggunakan Arrangement.End dan berada di tengah secara vertikal.

Pada baris ke [113] hingga [124], dibuat Button dengan lebar 100.dp dan tinggi 44.dp yang akan memanggil fungsi onClick saat ditekan. Tombol ini menggunakan warna latar onSurface dan teks berwarna onPrimary, serta label tombol ditulis “Detail” dengan ukuran font 16.sp menggunakan Text.

### **TopBarApp.kt**

Pada baris ke [1] terdapat deklarasi package com.pemrogramanmobile.movielist.presentation.ui.screen yang menunjukkan bahwa file ini berada dalam paket ui.screen dari modul presentation dalam aplikasi movielist, yang merupakan bagian dari struktur organisasi kode dalam proyek Android.

Pada baris ke [3] sampai [7], dilakukan impor berbagai komponen dari Jetpack Compose seperti ikon DarkMode dan LightMode dari material.icons.filled, serta berbagai elemen dari material3 seperti TopAppBar, Text, Icon, IconButton, dan MaterialTheme, yang semuanya diperlukan untuk membangun tampilan bar atas (Top App Bar) dengan ikon pengubah mode gelap dan terang.

Pada baris ke [9] digunakan anotasi @OptIn(ExperimentalMaterial3Api::class) yang berarti bahwa fungsi ini menggunakan fitur eksperimental dari Material3 API yang belum sepenuhnya stabil, sehingga diperlukan opt-in secara eksplisit untuk menghindari peringatan saat kompilasi.

Pada baris ke [10] sampai [15], dideklarasikan sebuah fungsi composable bernama TopBarApp yang menerima tiga parameter: title bertipe String sebagai judul yang akan ditampilkan di bar atas, isDarkMode bertipe Boolean untuk menentukan apakah saat ini sedang dalam mode gelap, dan onToggleTheme, sebuah lambda tanpa parameter yang akan dipanggil ketika ikon tema ditekan.

Pada baris ke [16] hingga [19], digunakan komponen TopAppBar dari Material3 untuk membangun tampilan bar atas. Pada parameter title, terdapat Text(text = title) yang akan menampilkan teks judul sesuai nilai parameter title.

Pada baris ke [20] sampai [27], didefinisikan bagian actions dari TopAppBar yang berisi sebuah IconButton dengan aksi onClick = onToggleTheme. Ketika tombol ditekan, lambda onToggleTheme akan dipanggil untuk mengubah tema. Di dalam IconButton,

terdapat Icon yang akan menampilkan ikon LightMode jika sedang dalam mode gelap (isDarkMode == true), dan ikon DarkMode jika sedang dalam mode terang.

Pada baris ke [28] sampai [32], terdapat parameter colors yang menggunakan TopAppBarDefaults.topAppBarColors untuk menetapkan skema warna dari TopAppBar, yaitu warna latar belakang diambil dari MaterialTheme.colorScheme.primary, warna teks judul dari onPrimary, dan warna ikon aksi juga dari onPrimary agar kontras dengan warna latar.

### **MainActivity.kt**

Pada baris ke [1] terdapat deklarasi package com.pemrogramanmobile.movielist.presentation.ui yang menunjukkan bahwa file ini berada di dalam paket bernama presentation.ui dari aplikasi movielist. Paket ini berfungsi sebagai wadah untuk menyusun file berdasarkan fitur tampilan (UI) aplikasi.

Pada baris ke [3] sampai [25], dilakukan impor berbagai komponen penting dari Android Jetpack dan library Compose seperti ComponentActivity, setContent, Material3, dan NavHost. Impor lainnya mencakup rememberNavController untuk navigasi antar layar, ViewModelProvider untuk menginisialisasi ViewModel, serta kelas-kelas dari folder data, domain, dan presentation untuk menghubungkan database, API, ViewModel, dan UI aplikasi.

Pada baris ke [27], terdapat deklarasi kelas MainActivity yang merupakan turunan dari ComponentActivity. Kelas ini berperan sebagai entry point utama dari aplikasi, di mana seluruh logika UI pertama kali dijalankan melalui fungsi onCreate.

Pada baris ke [29] terdapat override dari fungsi onCreate, yang akan dieksekusi saat MainActivity pertama kali dijalankan. Parameter savedInstanceState digunakan untuk menyimpan dan memulihkan status aplikasi jika dibutuhkan.

Pada baris ke [31] dibuat instance dari database lokal menggunakan MovieDatabase.getDatabase(applicationContext), yang akan mengakses atau membuat database SQLite melalui Room.

Pada baris ke [32] dibuat instance repository dari MovieRepositoryImpl yang menerima API Retrofit dan DAO dari database untuk melakukan operasi data baik secara lokal maupun jaringan.

Pada baris ke [33] dibuat instance useCase dari MovieUseCase, yang berfungsi sebagai perantara antara ViewModel dan Repository untuk mengatur logika bisnis aplikasi.

Pada baris ke [34] dideklarasikan movieViewModel dengan menggunakan ViewModelProvider dan MovieViewModelFactory. ViewModel ini mengatur data dan status tampilan utama dari daftar film di aplikasi.

Pada baris ke [36] dibuat instance `themeViewModel` menggunakan `ThemeViewModelFactory`, yang digunakan untuk mengatur dan menyimpan preferensi tema gelap atau terang berdasarkan status yang tersimpan di `DataStore`.

Pada baris ke [38] dipanggil fungsi `setContent` yang akan menggantikan isi tampilan aktivitas menggunakan `Compose`. Seluruh UI ditulis dalam blok `lambda` ini.

Pada baris ke [40] digunakan delegasi `by` untuk mengamati perubahan data `isDarkMode` dari `themeViewModel` secara otomatis menggunakan `collectAsState()` yang merupakan ekstensi `Compose` untuk `Flow`.

Pada baris ke [42] dipanggil fungsi `MovieListTheme` dengan parameter `darkTheme = isDarkMode`, yang akan mengatur apakah tampilan menggunakan tema gelap atau terang berdasarkan nilai boolean `isDarkMode`.

Pada baris ke [43] digunakan `Surface` sebagai wadah utama untuk seluruh konten UI, dan diberi warna latar `Color(0xFF131418)`, yaitu warna gelap yang akan digunakan sebagai `background` utama aplikasi.

Pada baris ke [44] dibuat objek `navController` menggunakan `rememberNavController()`, yang digunakan untuk mengatur navigasi antar layar dalam aplikasi.

Pada baris ke [45] hingga [66] digunakan `NavHost` dengan `navController` dan `startDestination` bernilai `"main"`, yang menunjukkan bahwa layar utama aplikasi adalah halaman utama. Di dalam `NavHost` terdapat dua rute: rute pertama `"main"` akan memanggil composable `MainScreen`, dan rute kedua `"detail/{id}"` akan memanggil composable `DetailScreen` dengan parameter `id` bertipe `integer`.

Pada baris ke [46] sampai [53], dipanggil `MainScreen` saat rute `"main"` diakses. Di dalamnya dikirim parameter `viewModel` untuk data film, `navController` untuk navigasi, `isDarkMode` untuk status tema saat ini, dan `onToggleTheme` sebagai `lambda` untuk mengganti tema saat tombol ditekan.

Pada baris ke [54] sampai [65] didefinisikan rute `"detail/{id}"` yang menerima argumen berupa `id` bertipe `IntType`. Argumen ini diambil dari `backStackEntry`, dan jika tidak ada maka diberi nilai default `-1`. Setelah itu, fungsi `DetailScreen` dipanggil dengan parameter `movieId`, `viewModel`, `isDarkMode`, dan `onToggleTheme`.

Pada baris ke [63] dipanggil `lambda onToggleTheme` yang akan mengeksekusi method `toggleTheme()` dari `themeViewModel` untuk mengganti mode tampilan gelap atau terang aplikasi.

## **MovieViewModel.kt**

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.presentation.viewmodel` yang berfungsi sebagai

penamaan paket tempat file ini berada dan mengelompokkan file ViewModel agar mudah dikelola sesuai struktur arsitektur aplikasi.

Pada baris ke [3] hingga [4], dilakukan impor ViewModel dan viewModelScope dari AndroidX Lifecycle yang diperlukan untuk membuat kelas ViewModel dan menjalankan coroutine di dalamnya agar tidak memblokir thread UI.

Pada baris ke [5] diimpor model data Movie dari domain model, yang mewakili entitas film yang digunakan dalam logika bisnis.

Pada baris ke [6] dilakukan impor MovieUseCase, yaitu class yang menyimpan logika pemanggilan data film dari repository yang digunakan oleh ViewModel.

Pada baris ke [7] diimpor NetworkResult, sebuah kelas pembungkus hasil jaringan (sukses, loading, error) yang berguna untuk menangani status pengambilan data dari API.

Pada baris ke [8] sampai [9], dilakukan impor MutableStateFlow dan StateFlow dari Kotlin Coroutines Flow, yang digunakan untuk menyimpan dan mengalirkan status data film ke UI secara reaktif.

Pada baris ke [10] diimpor launch dari Kotlin Coroutines untuk menjalankan fungsi secara asinkron di dalam coroutine scope milik ViewModel.

Pada baris ke [12], dibuat sebuah class bernama MovieViewModel yang merupakan turunan dari ViewModel, dengan parameter konstruktor movieUseCase yang bertipe MovieUseCase, digunakan sebagai perantara untuk mengambil data film dari repository.

Pada baris ke [13], dideklarasikan variabel privat \_movies bertipe MutableStateFlow yang menyimpan data berupa objek NetworkResult<List<Movie>>, dan langsung diinisialisasi dengan nilai Loading sebagai status awal.

Pada baris ke [14], dibuat properti publik movies bertipe StateFlow yang hanya menyediakan getter dari \_movies, agar data hanya bisa dibaca dari luar ViewModel namun tidak bisa diubah secara langsung.

Pada baris ke [16], terdapat blok init yang akan dipanggil saat objek MovieViewModel pertama kali dibuat, dan di dalamnya langsung memanggil fungsi fetchMovies() untuk memulai pengambilan data film.

Pada baris ke [20], dibuat fungsi fetchMovies() yang digunakan untuk mengambil data film populer dari MovieUseCase secara asynchronous.

Pada baris ke [21], fungsi ini menjalankan coroutine menggunakan viewModelScope.launch agar tetap terikat ke siklus hidup ViewModel dan tidak menyebabkan memory leak.

Pada baris ke [22], movieUseCase.getPopularMovies() dipanggil dan dikumpulkan (collect) sebagai aliran data Flow, yang setiap hasilnya akan diteruskan ke \_movies.value.

Pada baris ke [23], nilai hasil pengambilan film disimpan ke dalam \_movies.value, yang nantinya akan diamati oleh UI untuk ditampilkan berdasarkan status loading, sukses, atau error.

### **MovieViewModelFactory.kt**

Pada baris ke [1] terdapat package com.pemrogamanmobile.movielist.presentation.viewmodel yang menunjukkan bahwa file ini merupakan bagian dari struktur paket aplikasi yang mengatur komponen ViewModel pada layer presentasi.

Pada baris ke [3] dan [4], dilakukan impor class ViewModel dan ViewModelProvider dari AndroidX Lifecycle, yang dibutuhkan untuk membuat dan mengelola ViewModel secara dinamis, terutama saat menggunakan dependency injection manual tanpa framework seperti Hilt atau Dagger.

Pada baris ke [5] diimpor MovieUseCase, yaitu class yang bertanggung jawab menyediakan logika bisnis untuk pengambilan data film yang akan digunakan oleh ViewModel.

Pada baris ke [7], dibuat class MovieViewModelFactory yang merupakan turunan dari ViewModelProvider.Factory, dan berfungsi sebagai pabrik (factory) untuk membuat instance MovieViewModel dengan parameter useCase yang diteruskan melalui konstruktor.

Pada baris ke [8], dilakukan override terhadap fungsi create dari interface ViewModelProvider.Factory, dengan parameter modelClass bertipe Class<T>, untuk menentukan ViewModel apa yang akan dibuat berdasarkan tipe yang diminta.

Pada baris ke [9] sampai [11], dilakukan pengecekan apakah modelClass yang diminta merupakan turunan dari MovieViewModel. Jika benar, maka akan mengembalikan instance MovieViewModel yang dibuat menggunakan useCase yang sudah disediakan, dan di-casting menjadi tipe generik T.

Pada baris ke [12] dibuat pengecualian menggunakan throw IllegalArgumentException dengan pesan “Unknown ViewModel class” apabila modelClass yang diminta bukan MovieViewModel, sebagai penanganan jika ViewModel yang tidak dikenali diminta untuk dibuat.

### **ThemeViewModel.kt**

Pada baris ke [1] terdapat package com.pemrogamanmobile.movielist.presentation.viewmodel yang menunjukkan bahwa file ini termasuk dalam struktur modul presentasi aplikasi, khususnya digunakan untuk menyimpan dan mengelola logic ViewModel yang berkaitan dengan tampilan aplikasi.

Pada baris ke [3] sampai [5], dilakukan impor class ViewModel dari AndroidX Lifecycle untuk membuat class berbasis ViewModel, viewModelScope untuk menjalankan coroutine di dalam lifecycle ViewModel, serta PreferencesManager yang merupakan class kustom untuk mengelola penyimpanan preferensi tema gelap atau terang.

Pada baris ke [6] dan [7], diimpor MutableStateFlow dan StateFlow dari Kotlin Coroutines Flow, yang digunakan untuk menyimpan dan mengamati status mode tema secara reaktif agar dapat terintegrasi dengan UI secara otomatis.

Pada baris ke [10] dideklarasikan class ThemeViewModel yang merupakan turunan dari ViewModel, dengan konstruktor yang menerima parameter preferencesManager bertipe PreferencesManager untuk membaca dan menyimpan status tema dari penyimpanan lokal.

Pada baris ke [12] dibuat variabel privat `_isDarkMode` bertipe `MutableStateFlow` dengan nilai awal yang diambil dari fungsi `preferencesManager.isDarkMode()`, yang mengembalikan apakah saat ini aplikasi menggunakan mode gelap atau tidak.

Pada baris ke [13] dideklarasikan variabel publik `isDarkMode` bertipe `StateFlow<Boolean>` yang merepresentasikan status terkini mode tema dan hanya bisa dibaca dari luar class, sedangkan perubahan hanya dapat dilakukan dari dalam `ViewModel`.

Pada baris ke [15] sampai [21] dibuat fungsi `toggleTheme()` yang akan membalikkan status tema saat ini. Pertama, nilai mode saat ini dibalik, lalu disimpan ke `_isDarkMode`. Kemudian, di dalam coroutine menggunakan `viewModelScope.launch`, perubahan tema tersebut juga disimpan secara permanen melalui fungsi `preferencesManager.saveThemeSetting(newMode)`.

### **ThemeViewModelFactory.kt**

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.presentation.viewmodel` yang menunjukkan bahwa file ini berada dalam direktori presentasi pada struktur aplikasi dan berfungsi untuk menyimpan logika `ViewModel` yang berkaitan dengan tampilan antarmuka pengguna.

Pada baris ke [3] sampai [6], dilakukan impor class `Context` dari `Android`, `ViewModel` dan `ViewModelProvider` dari `AndroidX Lifecycle`, serta class `PreferencesManager` dari package `data.preferences`, yang keseluruhannya diperlukan untuk membuat dan menyediakan `ViewModel` dengan dependensi tambahan berupa konteks aplikasi.

Pada baris ke [8] dideklarasikan class `ThemeViewModelFactory` yang mengimplementasikan interface `ViewModelProvider.Factory`, dengan satu parameter konstruktor berupa `context` yang digunakan untuk menginisialisasi class `PreferencesManager`.

Pada baris ke [9] sampai [12] dibuat override fungsi `create()` yang menerima parameter `modelClass` bertipe `Class<T>` untuk menentukan tipe `ViewModel` yang ingin dibuat. Di dalam fungsi ini, objek `prefs` bertipe `PreferencesManager` diinisialisasi menggunakan `context` yang diberikan.

Pada baris ke [11] nilai `prefs` kemudian digunakan untuk membuat instance dari `ThemeViewModel`, yang kemudian dikembalikan dengan melakukan casting ke tipe `T` agar sesuai dengan parameter generik fungsi `create`.

### **NetworkResult.kt**

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.utils` yang menunjukkan bahwa file ini berada dalam direktori utilitas dari aplikasi, dan berfungsi untuk menyimpan class-class bantu atau kelas utilitas seperti pengelola hasil dari permintaan jaringan.

Pada baris ke [3] dideklarasikan class `NetworkResult` sebagai sealed class dengan parameter generik `T`, yang berarti hanya subclass yang didefinisikan di dalam file ini yang

dapat menjadi turunan dari `NetworkResult`. Ini berguna untuk membatasi hasil respons jaringan agar hanya memiliki beberapa kemungkinan keadaan yang spesifik.

Pada baris ke [4] dibuat subclass `Success` sebagai data class dengan parameter data bertipe generik `T`, yang mewakili keadaan ketika permintaan jaringan berhasil dan mengembalikan data.

Pada baris ke [5] didefinisikan subclass `Error` sebagai data class dengan parameter message bertipe `String`, yang menunjukkan bahwa permintaan jaringan gagal dan mengembalikan pesan kesalahan.

Pada baris ke [6] terdapat deklarasi object `Loading` yang merupakan turunan dari `NetworkResult` dan menandakan bahwa proses permintaan jaringan sedang berlangsung tanpa membawa data apapun.

### **Tautan Git**

Berikut adalah tautan untuk source code yang telah dibuat.

[https://github.com/Harry154git/Pemrograman\\_mobile/tree/main/Modul 5](https://github.com/Harry154git/Pemrograman_mobile/tree/main/Modul%205)